State Management, Part 2

BLoC, Cubit, repositories, testing, and Riverpod

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Autumn 2026

TODAY

What you should leave with

Events in, states out

Write a Cubit and a Bloc, and know which of the two a screen deserves.

The widgets

BlocProvider, BlocBuilder, BlocListener, BlocConsumer, read and watch.

State and data

Immutable Equatable states, copyWith, and a repository under the bloc.

Prove it works

Test a bloc with bloc_test, and read Riverpod code when you meet it.

PART 1 · WHY

When setState is not enough

Shared state, side effects, and tests

WHY

Where setState stops scaling

Two screens, one truth

The list screen and the detail screen both own a copy. They drift apart within a week.

Side effects

A snack bar, a navigation, an analytics event. Firing them from build runs them on every rebuild.

Testing

To test logic inside a State object you must pump a widget. Logic outside the tree is tested in milliseconds.

Ordering

Two taps start two requests. The slower one answers last and overwrites the newer data.

No history

setState leaves no trace. You cannot see what changed, in which order, or why.

Rebuild scope

setState rebuilds the whole State subtree, even the parts that did not change.

WHY

The shape of the answer

The UI sends

A tap becomes an event or a method call. The widget does not decide anything.

The bloc decides

It holds the current state, talks to repositories, and emits the next state.

The UI renders

Every new state is a stream item. Widgets that listen rebuild from it. Nothing else does.

BLoC stands for Business Logic Component. The `flutter_bloc` package gives you the base classes and the widgets that connect them to the tree.

One direction only. Events go up, states come down. When you can point at the single line that produced a state, debugging stops being guesswork.

WHY

The vocabulary

TERM	WHAT IT IS	IN CODE
State	An immutable snapshot of a feature	a class with final fields
Event	Something that happened, as an object	`TodoAdded('milk')`
Cubit	An object with methods that emit states	`extends Cubit<S>`
Bloc	A Cubit driven by events instead	`extends Bloc<E,S>`
emit	Publish the next state	`emit(next)`
Provider	Puts the instance in the widget tree	`BlocProvider`

A Cubit is a Bloc without events. Both expose the current state and a stream of states.

PART 2 · CUBIT AND BLOC

Two classes, one pattern

emit, on<Event>, and how to choose

A Cubit is a class with methods

```dart
class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() => emit(state + 1);
  void decrement() => emit(state - 1);
  void reset() => emit(0);
}
```

Three things to see.
`super(0)` is the initial state. `state` always reads the current one. `emit` publishes the next one.
The state type is any Dart type. Here it is `int`. For a real feature it is a class, which the next part covers.

No Flutter imports in this file. The Cubit knows nothing about widgets, so it runs in a plain Dart test with no emulator.

Providing it, and reading it

```
BlocProvider(
  create: (context) => CounterCubit(),
  child: const CounterPage(),
)

// Anywhere below it in the tree:
BlocBuilder<CounterCubit, int>(
  builder: (context, count) => Text('$count'),
)

onPressed: () =>
    context.read<CounterCubit>().increment(),
```

Two halves.
`BlocProvider` owns the instance and closes it when the widget is removed.
`BlocBuilder` subscribes to the stream and rebuilds only its own builder, not the whole page.

Order

The provider must sit above the widget that reads it, never beside it.

A Cubit that loads data

```
class TodoCubit extends Cubit<TodoState> {
  TodoCubit(this._repo) : super(const TodoState());
  final TodoRepository _repo;

  Future<void> load() async {
    emit(state.copyWith(status: Status.loading));
    try {
      final todos = await _repo.fetchTodos();
      emit(state.copyWith(status: Status.ready, todos: todos));
    } catch (_) {
      emit(state.copyWith(status: Status.failure));
    }
  }
}
```

Bloc: events are objects

```
sealed class TodoEvent extends Equatable {
  const TodoEvent();
  @override
  List<Object?> get props => const [];
}
final class TodosRequested extends TodoEvent {}
final class TodoAdded extends TodoEvent {
  const TodoAdded(this.title);
  final String title;
  @override
  List<Object?> get props => [title];
}
```

Name events in the past tense.

They describe what happened, not what to do. `TodoAdded`, not `addTodo`. Events carry data as final fields. Equatable events can be compared in a test and deduplicated by the bloc.

The Bloc: one handler per event

```
class TodoBloc extends Bloc<TodoEvent, TodoState> {
  TodoBloc(this._repo) : super(const TodoState()) {
    on<TodosRequested>(_onRequested);
    on<TodoAdded>(_onAdded);
  }
  final TodoRepository _repo;
  Future<void> _onRequested(
      TodosRequested event, Emitter<TodoState> emit) async {
    emit(state.copyWith(status: Status.loading));
    final todos = await _repo.fetchTodos();
    emit(state.copyWith(status: Status.ready, todos: todos));
  }
}
```

Cubit or Bloc

	CUBIT	BLOC
Input	method call	event object added with `add`
Boilerplate	one class	one class plus an event hierarchy
Traceable	the method name	every input is a value you can log or replay
Concurrency	none built in	transformers: droppable, restartable
Good for	a form, a toggle, a counter	search, pagination, anything with races

Start with a Cubit. Promote it to a Bloc the day you need to log the inputs, debounce them, or drop the ones that arrive while a request runs.

PART 3 · THE WIDGETS

Connecting a bloc to the tree

Provider, builder, listener, consumer

BlocProvider: one owner, one subtree

```
MultiBlocProvider(
  providers: [
    BlocProvider(create: (c) =>
     AuthCubit(auth)),
    BlocProvider(create: (c) =>
     TodoBloc(repo)),
  ],
  child: const App(),
)

// Reuse an instance on a new route:
BlocProvider.value(
    value: bloc, child: const EditPage())
```

create versus value.
`create` builds the instance and closes it later. `value` borrows one that `context.read` already found.

⚠ Do not mix them

Passing an existing instance to `create` means it gets closed twice.

BlocBuilder and buildWhen

```
BlocBuilder<TodoBloc, TodoState>(
  buildWhen: (previous, current) =>
      previous.todos != current.todos,
  builder: (context, state) => ListView(
    children: [
      for (final t in state.todos)
        TodoTile(t),
    ],
  ),
)
```

By default it rebuilds on every state.

`buildWhen` narrows that to the states this widget actually cares about.

Keep the builder small and put it as deep in the tree as possible. A BlocBuilder around a whole Scaffold rebuilds the whole Scaffold.

BlocListener: side effects, exactly once

```
BlocListener<TodoBloc, TodoState>(
  listenWhen: (previous, current) => previous.status != current.status,
  listener: (context, state) {
    if (state.status != Status.failure) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Load failed')),
    );
  },
  child: const TodoView(),
)
```

The listener runs once per state, so navigation, snack bars and analytics belong here. A builder can run many times for one state: after a resize or a parent rebuild.

BlocConsumer: both, when both are needed

```
BlocConsumer<AuthCubit, AuthState>(
  listenWhen: (previous, current) => current.error != null,
  listener: (context, state) => showErrorDialog(context, state.error!),
  buildWhen: (previous, current) => previous.user != current.user,
  builder: (context, state) =>
      state.user == null ? const LoginForm() : HomeView(state.user!),
)
```

One widget, two callbacks, filtered independently. Use it when the same state drives both a rebuild and a side effect. If only one of the two is needed, use the single-purpose widget.

WIDGETS

read, watch, select

CALL	GIVES YOU	USE IT
`context.read<T>()`	the instance, no subscription	in callbacks: onPressed, initState
`context.watch<T>()`	the state, and rebuilds	in build, when the whole widget depends on it
`context.select`	one field, and rebuilds on it	in build, when only one field matters
`BlocProvider.of<T>`	the same as read	older code, same thing

`select` takes a function and rebuilds only when its return value changes.

read in callbacks, watch in build. Calling watch outside build throws. Calling read in build gives a value that never updates.

PART 4 · STATE AND DATA

Designing state, and its data

Equatable, copyWith, repositories

One state class with a status

```
enum Status { initial, loading, ready, failure }

class TodoState extends Equatable {
  const TodoState({
    this.status = Status.initial,
    this.todos = const [],
  });
  final Status status;
  final List<Todo> todos;

  @override
  List<Object?> get props => [status, todos];
}
```

Every field is final.
A state is a snapshot. To change it you build a new one, you never edit the old one. `props` is the whole contract: `==` and `hashCode` are generated from exactly that list. Add a field, add it to props.

STATE

copyWith, and the two failure modes

```
TodoState copyWith({
  Status? status,
  List<Todo>? todos,
}) => TodoState(
      status: status ?? this.status,
      todos: todos ?? this.todos,
    );

// Wrong: same list object, so the new
// state compares equal and nothing happens.
state.todos.add(todo);
emit(state);
```

Build a new list.
The correct line emits a copy that holds a fresh list: `[...state.todos, todo]`.

⚠ Two mirrored bugs

No value equality: the UI re-builds when nothing changed. Mutation in place: the UI does not rebuild when the data did change.

Sealed states and an exhaustive switch

```
sealed class AuthState {}
final class AuthSignedOut extends AuthState {}
final class AuthSignedIn extends AuthState {
  AuthSignedIn(this.user);
  final User user;
}
Widget body(AuthState state) => switch (state) {
      AuthSignedOut() => const LoginView(),
      AuthSignedIn(:final user) => HomeView(user: user),
    };
```

Sealed means the compiler knows every case: add a third state and every switch that forgot it stops compiling. Fields live on the state that owns them, so no branch checks a nullable user.

Which shape to pick

One class with a status

Fewer classes, and copyWith carries data across a reload so the list stays on screen while it refreshes. Cost: states like loading with old data and an error must be checked by hand.

A sealed hierarchy

Impossible states cannot be built, and the switch is checked at compile time. Cost: more classes, and shared data is repeated on each state.

Both are used in production code. What you must not do is mix them inside one feature.

Pick one per feature and stay with it. A screen that keeps data visible while refreshing wants the status field. A screen with genuinely disjoint modes wants the sealed hierarchy.

DATA

The layers under a screen

Widget

Renders a state and sends events. Holds no business rule.

Bloc or Cubit

Owns the rules and the current state. Knows the repository, not the network.

Repository

One method per use case. Returns models, decides cache versus network.

Data source

The dio client, the Firestore reference, the sqflite database. Raw JSON or rows.

The bloc never imports http or dio. Each layer knows only the one below it, so a test can swap the data source. If a bloc file names a URL, the repository is missing.

A repository

```dart
class TodoRepository {
  TodoRepository(this._api);
  final TodoApi _api;

  Future<List<Todo>> fetchTodos() async {
    final json = await _api.getTodos();
    return json.map(Todo.fromJson).toList();
  }

  Future<void> save(Todo todo) =>
      _api.putTodo(todo.toJson());
}
```

Plain Dart, no Flutter.
It returns models, not responses, and it throws domain errors, not status codes.
The bloc is written against this class. In a test you pass a fake with the same methods and no network at all.

DATA

Wiring it at the top of the app

```
RepositoryProvider(
  create: (context) => TodoRepository(TodoApi()),
  child: BlocProvider(
    create: (context) => TodoBloc(
      context.read<TodoRepository>(),
    )..add(TodosRequested()),
    child: const TodoPage(),
  ),
)
```

Repositories above blocs.
The repository outlives any single screen, so it is provided once, near the root.
The cascade after the constructor fires the first event as soon as the bloc exists, which is the usual way to trigger the initial load.

Folder structure by feature

```
lib/
  todos/
    bloc/
      todo_bloc.dart
      todo_event.dart
      todo_state.dart
    data/
      todo_repository.dart
      todo_api.dart
    view/
      todo_page.dart
      todo_tile.dart
  main.dart
```

Group by feature, not by kind.
A folder named `blocs` at the root means every change touches five distant folders. One feature is one folder you could delete in a single command. That is a useful test of whether the boundaries are real.

PART 5 · TESTING AND ALTERNATIVES

Proving it works

bloc_test, and Riverpod as the alternative

bloc_test: one full example

```
blocTest<TodoBloc, TodoState>(
  'add then toggle emits two ready states',
  build: () => TodoBloc(FakeTodoRepository()),
  act: (bloc) => bloc
    ..add(const TodoAdded('milk'))
    ..add(const TodoToggled('1')),
  expect: () => const [
    TodoState(status: Status.ready, todos: [milk]),
    TodoState(status: Status.ready, todos: [milkDone]),
  ],
);
```

`build` creates the bloc, `act` sends events, `expect` lists the states in order.

What to test at each layer

Repository

Feed it a fake API. Assert that JSON becomes models and that a failure becomes the error your app expects.

Bloc

Feed it a fake repository. Assert the sequence of states for each event, including the failure path.

Widget

Wrap the view in `BlocProvider.value` with a bloc you control, and assert what is on screen.

The bloc layer gives the best return. It holds the rules, it has no dependency on the tree, and its tests run without an emulator. Lab 5 asks for one such test.

Riverpod: providers without BuildContext

```
void main() => runApp(const ProviderScope(child: App()));
final repoProvider = Provider((ref) => TodoRepository(TodoApi()));

class TodoList extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final todos = ref.watch(todoListProvider);
    return ListView(children: todos.map(TodoTile.new).toList());
  }
}
```

Providers are top-level values, so a missing one is a compile error, not a runtime exception.

NotifierProvider holds the logic

```
class TodoListNotifier extends Notifier<List<Todo>> {
  @override
  List<Todo> build() => const [];

  void add(Todo todo) => state = [...state, todo];
}

final todoListProvider =
    NotifierProvider<TodoListNotifier, List<Todo>>(TodoListNotifier.new);
```

The same rules apply here. State is replaced, never mutated. `ref.watch` in build subscribes, `ref.read` in a callback does not. Notifiers are tested exactly like cubits.

Choosing between them

	FLUTTER_BLOC	RIVERPOD
Lookup	by type, through the tree	by reference, top level
Missing one	throws at runtime	does not compile
Structure	events, states, handlers	providers you compose freely
Boilerplate	more, and very predictable	less, and more freedom to get lost
Best at	auditable flows, teams	dependency graphs and caching

This course uses flutter_bloc. Both are good. Pick the one your team already runs, and never run two state management packages in one app.

MISTAKES

Four mistakes to expect

Logic in the widget

An `if` in build that decides a business rule. Move it into the bloc and emit the decision as state.

Mutating state in place

Editing the list inside the state, then emitting the same object. Nothing rebuilds. Build a new list.

Side effects in a builder

A snack bar or a navigation inside `builder`. It fires again on every rebuild. Use a listener.

One giant bloc

An AppBloc with thirty events. Split it by feature, the same way the folders are split.

A fifth one, for completeness: calling `emit` after a handler has finished. The package throws a clear error for it.

RECAP

Three things to remember

1. Events in, states out. The widget sends and renders. The bloc decides. Nothing else mutates state.
2. State is immutable and compared by value. Equatable props decide whether the screen rebuilds at all.
3. The bloc depends on a repository. That boundary is what makes a five millisecond test possible.

FURTHER READING

bloclibrary.dev · pub.dev/packages/flutter_bloc · bloc_test · riverpod.dev

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel